AI Package Damage Inspector

Technical report: architecture, algorithms, decision logic, evaluation and limitations

Image & Video Processing (AIDLL 5052) mini project

ACCEPT package is fine **REJECT** crushed, torn or stained **NEEDS REVIEW** unsure or bad photo: a person checks

Abstract

This project inspects photos of parcels and decides whether to **ACCEPT** or **REJECT** them, or to send them for **MANUAL REVIEW**. It classifies damage as crushed, torn or water-stained, locates it, estimates severity and explains every decision. Two engines share one decision layer: a classical image-processing pipeline with a Random Forest and a CNN, trained on procedurally generated packages, and a real-photo engine that combines U^2-Net segmentation with the CLIP vision-language model used zero-shot. On 200 held-out synthetic images the full system reaches 99.5% accuracy on decided packages with 0.7% false acceptances, 0.0% false rejections and 2.5% sent to review. On real photos the synthetic-trained models fail completely (domain shift), which motivated the CLIP engine; its real-photo performance is reported honestly as a calibration result, because no independent labelled real dataset was available.

Contents

1	Problem and motivation
2	System architecture
3	Input validation
4	Real-photo engine
5	Synthetic-package engine
6	Decision layer
7	Visual explanations
8	Application and reporting
9	Evaluation
10	Limitations and future work
11	Running and demonstrating the project
A	Example JSON report

1. Problem and motivation

Parcels are damaged in transit: boxes get crushed, torn or wet. Checking every parcel by eye is slow and inconsistent. The goal is an inspection station that looks at a photo and makes a consistent, explainable decision, and that knows when it is unsure.

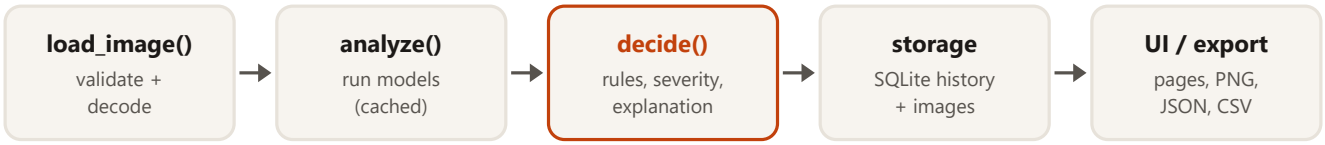
Requirements

- Decide ACCEPT / REJECT / NEEDS REVIEW from one photo.
- Name the damage type (intact, crushed, torn, stained) and detect photos without a package.
- Locate the package and the suspected damage; estimate severity; explain the decision.
- Never invent numbers: if something was not measured, say so.
- Work on ordinary photos (phone, webcam), not only on lab images.

Why two engines: the domain-shift problem

The first version of the project generated 1,600 synthetic packages (boxes with tape, labels, dents, holes and water stains drawn with OpenCV) and trained a Random Forest and a CNN on them. These models work very well on synthetic images, but on real photos they collapse: on the bundled real photos they classify 0.0% of the packages correctly, and during development they called every intact box damaged. Real photos differ in lighting, background, camera, printing and texture, so they fall outside the training distribution. Collecting and labelling thousands of real damaged parcels was not possible, so real photos are handled by a **foundation model** (CLIP) that has already learned from billions of real images, combined with classical image processing.

2. System architecture



The expensive step (`analyze`) only produces model outputs and is cached, so moving the threshold slider re-runs only the cheap `decide` step. Every page (single, batch, conveyor, evaluation) calls the same functions, so results are consistent everywhere.

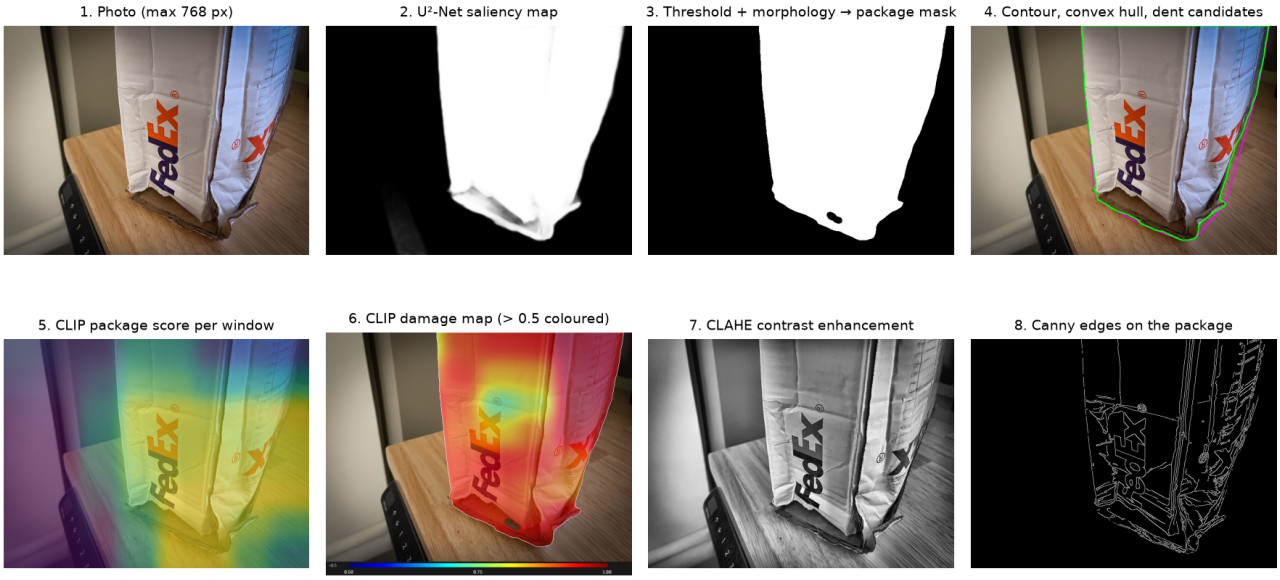
Module	Responsibility
inspection.py	Single source of truth: upload validation, analysis, decision rules, explanations, export
real_inspector.py	Real-photo engine: U^2-Net, CLIP scoring, damage map, dents, image quality, drawing
damage_detector.py	Synthetic generator, classical features, Random Forest + CNN, rule-based localisation
storage.py	Persistent inspection history (SQLite + annotated / original images on disk)
evaluation.py	Held-out synthetic set, real-photo dataset loader, metrics
ui.py, app.py, app_pages/	Streamlit interface (6 pages)

3. Input validation

Uploaded files are untrusted. Before any model runs, `load_image()` rejects empty files, files over 15 MB, formats other than JPG / PNG / WEBP / BMP (checked from the file header, not the extension), images above 40 megapixels (checked before decoding, against decompression bombs) and images smaller than 32 px. The

image is then decoded with OpenCV, which also applies the phone's EXIF rotation. Every failure produces a readable message, for example *"photo.gif" is a GIF file. Supported formats: JPG, PNG, WEBP, BMP.* Stored files are named after the inspection ID only, never after user input.

4. Real-photo engine



Intermediate steps for the crushed FedEx box: (1) photo, (2) U²-Net saliency, (3) thresholded + morphologically cleaned package mask, (4) contour, convex hull and dent candidates, (5) CLIP "is this a box?" score per window, (6) CLIP damage map, (7) CLAHE, (8) Canny edges on the package.

4.1 Package segmentation (U²-Net)

U²-Net-p, a 4.5 MB salient-object detection network, is run through the OpenCV DNN module on a 320×320 copy of the photo. Its saliency map is thresholded at 0.5, cleaned with a morphological opening and closing (9×9 elliptical kernel) and the largest connected component is kept as the package mask. If less than 6 % of the image is salient (typically a close-up where the box fills the frame), the whole frame is used and the report says so.

4.2 Zero-shot classification with CLIP

CLIP maps images and sentences into the same 512-dimensional space. For each class the system averages the text embeddings of 5 descriptions × 4 templates (prompt ensembling), for example:

Class	Example descriptions
intact	an intact cardboard box · an undamaged shipping box in good condition · a perfect sealed carton
crushed	a crushed cardboard box · a parcel with a smashed, caved-in corner · a crumpled, battered parcel
torn	a torn cardboard box · a ripped shipping box with a hole · a punctured cardboard box
stained	a water damaged cardboard box · a wet, stained shipping box · a soggy, discoloured cardboard box

Class probabilities are the softmax of 100 × cosine similarity. To catch damage that is small in the frame, the image is scored three ways and fused: **whole image** + **mirrored copy** (test-time augmentation), and **16 overlapping windows** (45 % of the image) weighted by how much each window looks like a box:

$$p = 0.5 \cdot p_{global} + 0.5 \cdot \Sigma\, w_i \cdot p_i \,/\, \Sigma\, w_i,\; w_i = \min(1,\, 1.5 \cdot \text{package_score}_i)$$

4.3 Package gate

The same model compares the photo with 6 package descriptions against 14 other-object descriptions (a person, a building, a street, a document with text, a telephone booth, ...). The softmax share of the package description is the **package score**. During development most non-package photos scored 0-5 %; a fire-alarm photo scored about 50 % and was therefore sent to review as "package uncertain".

4.4 Damage localisation

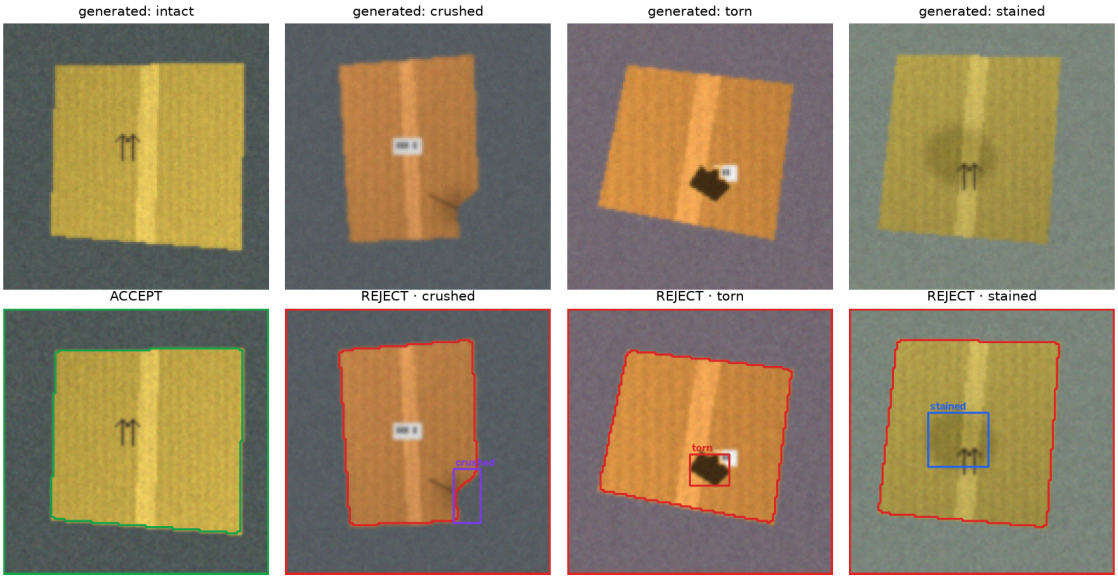
49 smaller windows (25 % of the image, 7×7 grid) are scored for P(damaged) = 1 − P(intact). Each window's score is spread over the pixels it covers, weighted by how much of the window lies on the package mask, smoothed, and set to zero outside the mask: this is the **damage map**. Regions where the map exceeds 0.5 become "suspected damage" boxes, each labelled with the damage type of the windows inside it (blended 50/50 with the global prediction). For crushed packages, **convexity defects** of the package outline deeper than 4.5 % of the package size are marked as dent candidates.

4.5 Image quality

Sharpness is the variance of the Laplacian and brightness the mean grey level, both measured at a fixed 512 px scale. During development, sharp real photos measured 136 to 7,500 and blurred copies at most 10; normal photos had brightness 86-185 and under-exposed copies 13-25. The thresholds (40 and 40-235) sit between these groups.

5. Synthetic-package engine

A procedural generator draws 128×128 packages: a perspective-distorted box with corrugation, paper grain, uneven lighting, packing tape, shipping labels with barcodes and "this side up" arrows, on a background that sometimes contains conveyor slats. Damage is drawn explicitly: a pushed-in corner with creases and shadow (crushed), irregular dark holes with frayed edges (torn), or blurred water marks with a tide-line ring (stained).



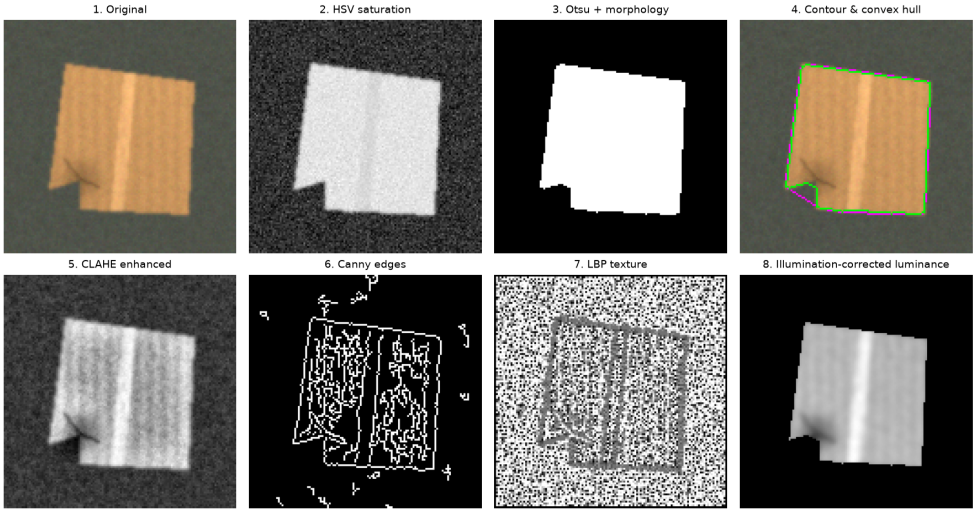
Top: generated packages. Bottom: inspection results from the synthetic engine, with damage regions found by the rule-based localiser.

5.1 Classical image-processing features (28)

Group	Features	Detects
Shape	area ratio, solidity, extent, vertex count, circularity, max / total convexity-defect depth	crushing (dented outline)

Group	Features	Detects
Dark regions	dark-pixel ratio, largest dark blob (V < 0.5 × median)	tears / holes
Illumination-corrected luminance	LAB lightness ÷ fitted lighting plane, share in 0.55-0.88	stains
Texture / edges	Canny edge density after CLAHE, Laplacian variance, 10-bin uniform LBP histogram	creases, frayed edges
Colour	HSV mean and standard deviation	discolouration

Segmentation uses Otsu thresholding on the HSV saturation channel, morphological opening and closing, and the largest contour. A **Random Forest** (300 trees) is trained on the 28 features and a **CNN** (four Conv-BatchNorm-ReLU-MaxPool blocks, 16-32-64-64 channels, global average pooling, dropout 0.3) on the raw pixels; their probabilities are averaged. Training uses 400 generated images per class (seed 42), 25 epochs, Adam with a one-cycle learning-rate schedule and random flips.



Classical IVP pipeline on a generated crushed package.

6. Decision layer

The decision layer (`inspection.decide`) is deterministic and identical for every page. Checks are applied in this order:



Rule	Setting	Reason
No package	package score < 0.35	not a parcel photo: retake
Package uncertain	package score < 0.70	cannot trust the classification
Image quality	sharpness < 40, brightness < 40 or > 235, short side < 160 px	blur and darkness make damage invisible or create false damage
Disagreement (real)	whole-image and local P(intact) on opposite sides of the threshold and > 0.30 apart	the two views of the same photo contradict each other
Disagreement (synthetic)	Random Forest and CNN predict different classes	models contradict each other
Too close to call	|P(intact) − threshold| < 0.05	a coin-flip decision is not acceptable
Accept / reject	P(intact) ≥ threshold → ACCEPT, else REJECT	threshold set in the app, default 0.65

Confidence, threshold and severity are separate

- **Confidence**: the model outputs (P(intact), P(damaged), class probabilities). Never modified.
- **Threshold**: a quality-control policy. Raising it makes inspection stricter. The default 0.65 was chosen because, on the development photos, CLIP gave intact boxes P(intact) of about 0.7 or more and damaged boxes less; accuracy stayed the same for any threshold from 0.60 to 0.75.
- **Severity**: the *extent* of the localised damage. Real photos: share of package windows with P(damaged) > 0.5 (< 40 % minor, < 75 % moderate, otherwise severe). Synthetic: share of package pixels inside damage regions (< 3 % / < 8 %). If no damage could be localised, severity is "not assessed".

Explanations

Each decision comes with plain-language reasons built from the values that produced it, for example for the FedEx box:

- *The model gives P(intact) = 28%, which is below the rejection threshold of 65%.*
- *Most likely damage type: crushed (49% of the damage probability).*
- *No review condition was triggered, so the package is rejected.*

7. Visual explanations

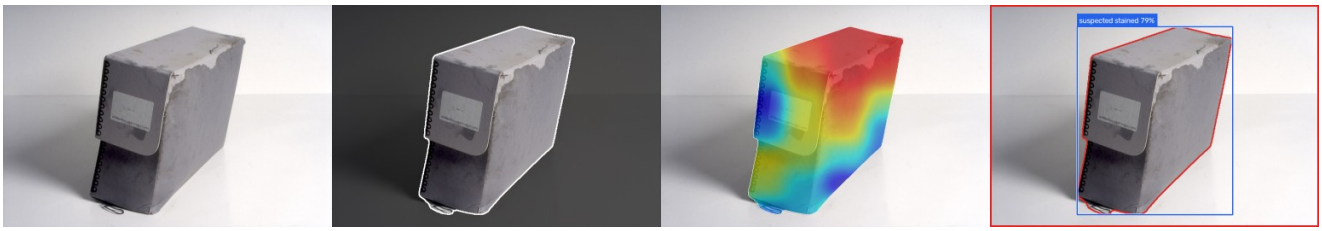
Each case below shows, left to right: the photo, the **package mask** (where the package is, from U²-Net), the **damage map** (where CLIP scores windows as damaged, only shown above 0.5) and the **result** (outline coloured by decision, suspected damage boxes coloured by type: violet crushed, red torn, blue stained).



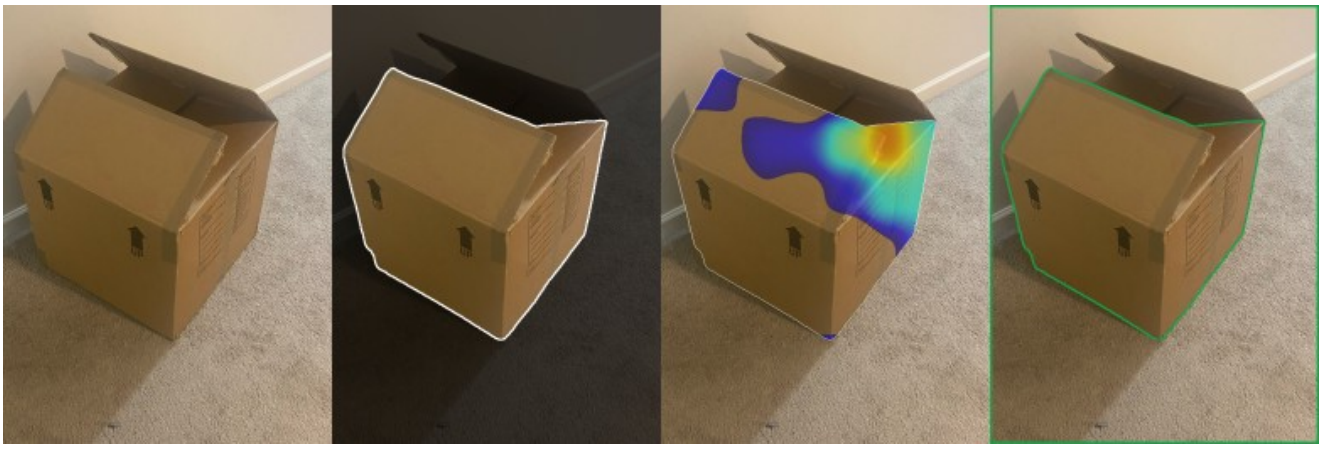
Damage-map colour scale: window P(damaged) from 0.5 (blue) to 1.0 (red); areas below 0.5 are not coloured.



crushed fedex box: REJECT, crushed, severe. P(intact) = 28%, damaged area 100 %.



stained archive box: REJECT, stained. P(intact) = 49%, damaged area 87 %.



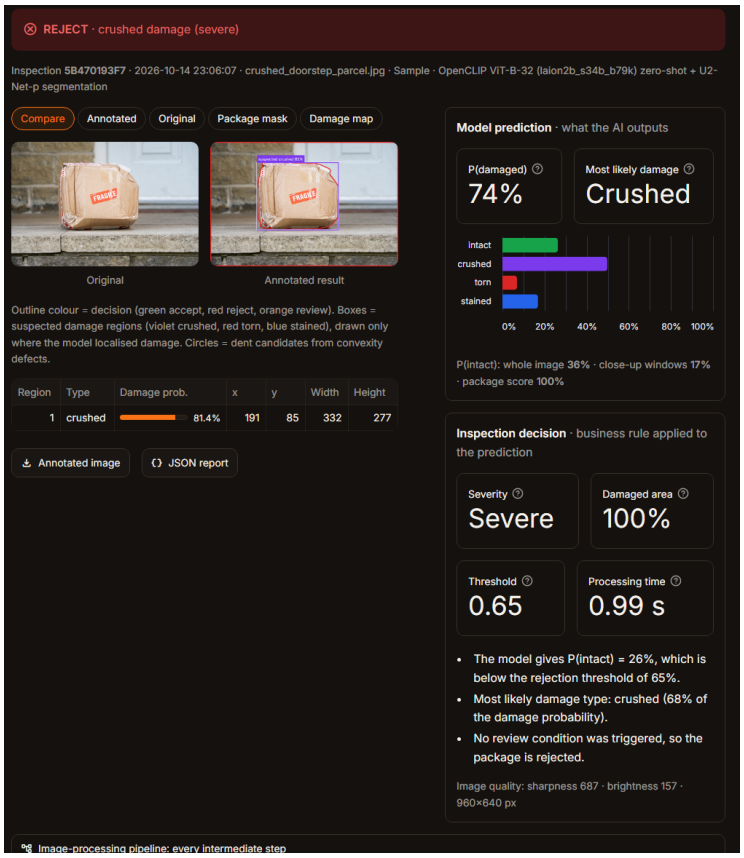
intact closed box: ACCEPT. A few windows score above 0.5 locally (blue), but the fused prediction is intact, so no damage regions are drawn and no severity is given. P(intact) = 78%.



not a package phone booth: NEEDS REVIEW: no package detected (package score 0 %), so no damage map is shown. P(intact) = 8%.

A separate view for each output keeps package segmentation and damage localisation from being confused, and the damage map uses a fixed probability scale so that a clean package is never painted red.

8. Application and reporting



Inspect page: decision banner, inspection ID and model, image views (compare / annotated / original / package mask / damage map), damage-region table, downloads, the model-prediction card and the inspection-decision card with severity, damaged area, threshold, measured processing time and explanation.

Feature	Implementation
Batch inspection	Up to 40 photos; each validated separately; failed files listed with their reason; CSV, ZIP of annotated PNGs, JSON
Conveyor simulation	Package enters → AI inspection → decision → routed to accept / reject / manual-review lane; clearly labelled as a software simulation

Feature	Implementation
Inspection history	SQLite database (data/inspections.db) + annotated and original images; filter by decision, type, source; search; detail view; CSV export; capped at 2,000 records
Reports	Annotated PNG and JSON per inspection (ID, timestamp, decision, reasons, type, severity, probabilities, threshold, timing, model, regions with their image size)
Robustness	Model failures and bad files produce readable errors; reruns reuse the same inspection, so the history has no duplicates; memory caches are bounded



Inspection history: filters, decision and damage-type charts, the stored inspections and the detail view of a selected inspection with its stored images and explanation.

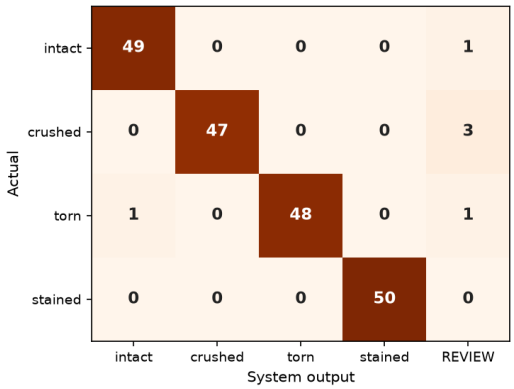
9. Evaluation

All results are produced by the full system (models and decision rules at threshold 0.65) on labelled images, with the same code as the app's Model evaluation page. Metrics: accuracy and per-class precision / recall / F1 on packages that received a decision; **false acceptance rate** = damaged packages accepted ÷ all damaged packages; **false rejection rate** = intact packages rejected ÷ all intact packages; **review rate** = share of package images sent to review.

9.1 Synthetic held-out set

200 images (50 per class) generated with seed 2026. The deployed models were trained on images generated with seed 42, so none of these images were seen during training; they do come from the same generator, so this measures performance within the synthetic domain only. (The 80/20 accuracy figures in the original notebook belong to the notebook's models; the deployed models were retrained on all generated images, so those figures are not reused here.)

99.5% accuracy (decided)	**99.5%** macro F1	**0.7%** false acceptance	**0.0%** false rejection
2.5% sent to review	**195/200** decided	**70 ms** mean time per image	**seed 2026** never seen in training



Class	Precision	Recall	F1	Images
intact	98.0%	100.0%	99.0%	49
crushed	100.0%	100.0%	100.0%	47
torn	100.0%	98.0%	99.0%	49
stained	100.0%	100.0%	100.0%	50

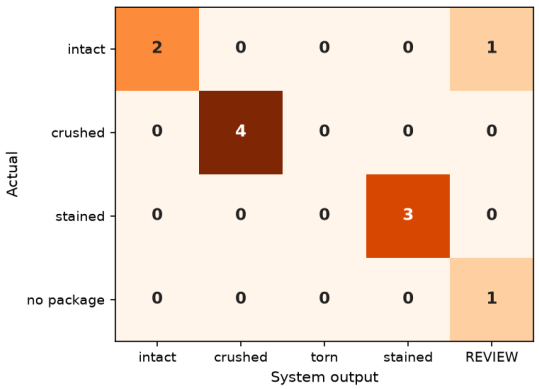
Confusion matrix (rows = actual class, columns = system output, including NEEDS REVIEW) and per-class performance on the synthetic held-out set.

9.2 Real photos

Dataset: 11 bundled sample photos (samples/): 4 crushed, 3 intact, 3 stained, 1 photo without a package (no real torn-box photos were available). **This is not an independent test:** the real-photo thresholds and rules were calibrated on these and similar photos, so the numbers are optimistic. The app evaluates any labelled photos placed in `eval_data/<class>/` instead, which is how an independent test should be done.

9/9 decided packages correct	**0.0%** false acceptance	**0.0%** false rejection	**10.0%** sent to review

100.0%	0.0%	1.3 s	11
non-packages caught	synthetic-trained RF + CNN	mean time per photo (CPU)	photos



The one review case is the intact packed box, whose only available version is 190×127 px: it is sent to review for low resolution rather than guessed. The synthetic-trained models classify none of the real package photos correctly, which quantifies the domain shift described in section 1.

9.3 Development history on real photos

During development, 23 Creative-Commons photos (8 intact, 5 crushed, 4 stained, 5 without a package and one historical photo of torn parcels being handled) were used to choose the method. Plain zero-shot CLIP (highest probability wins) got 15 of 23 right; adding a calibrated intact threshold and the global + local fusion raised this to 22 of 23. These photos guided the design, so they are calibration data, not a test set. Robustness check: blurred (Gaussian σ = 6) and darkened (×0.15) copies of the 13 package photos were all sent to NEEDS REVIEW with a quality reason.

9.4 Larger CLIP models

ViT-B/16 SigLIP2 and ViT-L/14 (DataComp, OpenAI) were also tried. The larger DataComp ViT-L/14 separated intact from damaged as well, but is about 3× larger and 6× slower on CPU; ViT-B/32 (LAION-2B) with fusion matched or exceeded it on the development photos at about 1 s per photo, so it was kept.

10. Limitations and future work

- **No independent real test set.** The most important next step is to photograph 100+ real parcels per class, put them in eval_data/ and measure the real false acceptance / rejection rates.
- **CLIP is not a damage detector.** It recognises "a crushed box" as a concept; subtle dents, damage on a far-away box or on a hidden side, and unusual packaging (bags, envelopes) are hard. With labelled real photos, a small classifier trained on CLIP image embeddings (a linear probe) would very likely improve accuracy.
- **Coarse localisation.** The damage map has window resolution (25 % of the image). A segmentation model trained on annotated damage would be needed for pixel-accurate damage outlines. Individual regions can be typed differently from the overall damage type.
- **Torn packages** are only tested on synthetic data; no real torn-box photos were available.
- **CPU speed.** About 1 s per real photo on a laptop CPU; a CUDA build of PyTorch would make it much faster.
- **Simulation.** The conveyor demonstrates the workflow only; no camera or belt hardware is connected.

11. Running and demonstrating the project

Step	Command / action
Install (first time)	setup.bat - installs requirements.txt and downloads the CLIP model (~600 MB)
Run	run_app.bat (or py -3.14 -m streamlit run app.py), open http://localhost:8501
Requirements	Python 3.14, streamlit, opencv-python, numpy, pandas, altair, torch, torchvision, scikit-learn, joblib, open_clip_torch, pillow, huggingface_hub

Suggested demonstration (about 6 minutes)

- **Inspect** → **Sample photos** → **Fedex box**: REJECT, crushed. Show Compare, Package mask vs Damage map, the prediction card vs the decision card and the explanation.
- **Closed box** (ACCEPT), **Phone booth** (NEEDS REVIEW, no package), **Packed box** (NEEDS REVIEW, too low resolution).
- Move the **rejection threshold** to 0.85 and show the decision changing; open the processing pipeline.
- **Camera**: a real box from the room; squash a corner for REJECT; shake the camera for the blur review.
- **Batch** → inspect all samples → export; **Conveyor** → run; **History** → filter and open a row.
- **Model evaluation**: run the synthetic held-out set and the real-photo domain-shift check.

Appendix A. Example JSON report

Exported by the JSON report button for the FedEx box inspection (abridged where marked).


```
{
  "id": "693B02BE71",
  "timestamp": "2026-10-14T23:29:32",
  "filename": "crushed_fedex_box.jpg",
  "source": "Sample",
  "decision": "REJECT",
  "review_reasons": [],
  "explanation": [
    "The model gives P(intact) = 28%, which is below the rejection threshold of 65%.",
    "Most likely damage type: crushed (49% of the damage probability).",
    "..."
  ],
  "invalid_image": false,
  "damage_type": "crushed",
  "type_share": 0.494,
  "severity": "Severe",
  "damaged_area_pct": 100.0,
  "area_method": "share of package windows with CLIP P(damaged) > 0.5",
  "p_intact": 0.275,
  "p_damaged": 0.725,
  "probabilities": {
    "intact": 0.275,
    "crushed": 0.358,
    "torn": 0.163,
    "stained": 0.204
  },
  "top_class": "crushed",
  "top_class_probability": 0.358,
  "threshold": 0.65,
  "review_margin": 0.05,
  "package_score": 0.999,
  "agreement": {
    "whole_image_p_intact": 0.319,
    "local_windows_p_intact": 0.23
  },
  "quality": {
    "sharpness": 581.4,
    "brightness": 116.8,
    "width": 960,
    "height": 720
  },
  "regions": [
    {
      "type": "crushed",
      "score": 0.902,
      "box": [278, 0, 471, 539]
    }
  ],
  "region_frame": [768, 576],
  "dents": [],
  "localisation_note": null,
  "notes": [],
  "analysis_ms": 938.5,
  "processing_ms": 957.0,
  "engine": "real-photo",
  "model": "OpenCLIP ViT-B-32 (laion2b_s34b_b79k) zero-shot + U2-Net-p segmentation",
  "annotated_image": "693B02BE71_annotated.png"
}
```